

## UNIT 3

### Multithreading In Java

#### ➤ THREAD

A thread in Java is the **smallest unit of execution** within a program.

It is a **single sequential flow of control** within a program.

They are **lightweight sub\_processes** that exist within a process.

Each thread has its own **local variable, program counter and lifetime**.

**Multithreading** in Java is a **process of executing multiple threads simultaneously**.

Multithreading allows **concurrent execution of two or more parts of a program** for **maximum utilization of CPU**.

#### ➤ Creating Threads

A new thread can be created in two ways.

##### A. By extending Thread class:

Define a class that extends Thread class and override its run() method with the code required by the thread.

**Syntax: -**

```
class Mythread extends Thread
{
    -----
}
```

**B. By implementing Runnable Interface:** Define a class that implements Runnable interface. The Runnable interface has only one method run(), that is to be defined in the method with the code to be executed by the thread.

**Syntax:**

```
class MyThread implements Runnable
{
    public void run()
    {
        -----
    }
}
```

## A] Extending Thread class

**Thread** class in **java.lang** package provide **constructors** and **methods** to **create** and **perform operations** on a thread.

When you extend this class, your subclass inherits all the properties and methods of `java.lang.Thread`.

### ➤ Steps to Extend the Thread Class

#### 1. Declare the class as extending the Thread class.

```
class MyThread extends Thread
{
    .....
}
```

Now we have a new type of thread `MyThread`.

#### 2. Implementing the `run()` Method

When we extend the class `Thread`, we need to implement the method `run()`

The `run` method is the **heart** and **soul** of any thread.

It makes the entire **body** of **thread** and is the only method in which the **thread's behaviour** can be implemented.

```
public void run()
{
    // Thread code
}
```

#### 3. Starting New Thread

Create an object of our new class and call `start()` method to start the execution of a thread. `start()` invokes the `run()` method on the `Thread` object.

```
MyThread obj = new MyThread( );
obj.start( ); //invokes run() method
```

#### An Example of Using the Thread Class

```
class A extends Thread
{
    public void run( )
    {
        for (int i=1; i<=5; i++)
        {
            System.out.println("\tFrom ThreadA : i = " + i);
        }
    }
}
```

```

System.out.println("Exit form A ");
}
}
class B extends Thread
{
public void run( )
{
for (int j=1; j<=5; j++)
{
System.out.println("\tFrom Thread B :j = " + j);
}
System.out.println("Exit from B ");
}
}
class C extends Thread
{
public void run( )
{
for(int k=1; k<=5; k++)
{
System.out.println("\tFrom Thread C : k = " + k);
}
System.out.println("Exit from C ");
}
}
class ThreadTest
{
public static void main(String args[ ])
{
A a1 = new A();
B b1= new B();
C c1=new C();
a1.start();
b1.start();
c1.start();
}
}

```

### Output of Program:

First run

|               |   |   |   |   |
|---------------|---|---|---|---|
| From Thread A | : | i | = | 1 |
| From Thread A | : | i | = | 2 |
| From Thread B | : | j | = | 1 |
| From Thread B | : | j | = | 2 |
| From Thread C | : | k | = | 1 |
| From Thread C | : | k | = | 2 |
| From Thread A | : | i | = | 3 |
| From Thread A | : | i | = | 4 |
| From Thread B | : | j | = | 3 |
| From Thread B | : | j | = | 4 |
| From Thread C | : | k | = | 3 |
| From Thread C | : | k | = | 4 |

```

From Thread A : i = 5
Exit from A
From Thread B : j = 5
Exit from B
From Thread C : k = 5
Exit from C

Second run
From Thread A : i = 1
From Thread A : i = 2
From Thread C : k = 1
From Thread C : k = 2
From Thread A : i = 3
From Thread A : i = 4
From Thread B : j = 1
From Thread B : j = 2
From Thread C : k = 3
From Thread C : k = 4
From Thread A : i = 5
Exit from A
From Thread B : k = 4
From Thread B : j = 5
From Thread C : k = 5
Exit from C
From Thread B : j = 5
Exit from B

```

Note that the outputs from the threads are not especially sequential. They do not follow any specific order. They are running independently of one another and each executes whenever it has a chance.

**Write a program to create two threads, one to print numbers in original order and other in reverse order from 1 to 10.** [8 M][W 14][S 17]

```

class thread1 extends Thread
{
public void run()
{
for(int i=1 ; i<=10;i++)
{
System.out.println("Thread 1 : " +i);
}
}
}
class thread2 extends Thread
{
public void run()
{
for(int i=10 ; i>=1;i--)
{
System.out.println("Thread 2 : " +i);
}
}
}
class test
{
public static void main(String args[])
{
thread1 t1 = new thread1();
thread2 t2= new thread2();
t1.start();
t2.start();
}
}

```

**Program : Create two thread , one to print even nos and another to print odd nos.**

```
class Even extends Thread
{
    public void run()
    {
        for (int i=1;i<=20;i++)
        {
            if(i%2==0)
            {
                System.out.println("From Thread Even : i = "+i);
            }
        }
        System.out.println("Exit from Thread Even");
    }
}

class Odd extends Thread
{
    public void run()
    {
        for(int j=1;j<=20;j++)
        {
            if(j%2==1)
            {
                System.out.println("From Thread Odd : j = "+j);
            }
        }
        System.out.println("Exit from Thread Odd");
    }
}

class Test
{
    public static void main(String[] args)
    {
        Even E = new Even();
        Odd O = new Odd();
        E.start();
        O.start();
    }
}
```



## B] Runnable interface in Java

Runnable interface in Java is used to make a **class instance** run as a **thread** by implementing it.

The Runnable interface is provided by the **java.lang** package.

It contains a **single abstract method, run()**, that will be executed when the thread is started.

### Declaration of Runnable interface :

```
public interface Runnable
{
    public abstract void run();
}
```

### Steps to Create a New Thread Using Runnable Interface in Java

1. Create a class that implements the Runnable interface.
2. Override the run() method of the Runnable interface in the class.
3. Create object of **Thread** class in the main class and passing the object of our class that implemented the Runnable interface.
4. Call the start() method using Thread object.

Java does not allow multiple inheritance, so if you extend from **Thread**, you can not extend from any other class.

By Implementing Runnable in our target class, we can still extend from other class.

### Q] Write a thread program for implementing the "Runnable interface". [8 M] [W 16]

//program to print even numbers from 1 to 20 using Runnable Interface

class mythread implements Runnable

```
{
    public void run()
    {
        System.out.println("Even numbers from 1 to 20 : ");
        for(int i= 1 ; i<=20; i++)
        {
            if(i%2==0)
                System.out.print(i+ " ");
        }
    }
}
```

class test

```
{
    public static void main(String args[])
    {
        mythread mt = new mythread();
        Thread t1 = new Thread (mt);
        t1.start();
    }
}
```

**Q] Write a program to print even numbers from 1 to 20 in one thread and print odd numbers between 1 to 20 in another thread, using Runnable Interface.**

**class mythread1 implements Runnable**

```
{
public void run()
{
System.out.println("Even numbers from 1 to 20 : ");
for(int i= 1 ; i<=20; i++)
{
if(i%2==0)
System.out.print(i+ " ");
}
}
}
```

**class mythread2 implements Runnable**

```
{
public void run()
{
System.out.println("Odd numbers from 1 to 20 : ");
for(int i= 1 ; i<=20; i++)
{
if(i%2==1)
System.out.print(i+ " ");
}
}
}
```

**class Sample**

```
{
public static void main(String args[])
{
mythread1 mt1 = new mythread1();
mythread2 mt2 = new mythread2();
```

```
Thread t1 = new Thread (mt1);
t1.start();
Thread t2 = new Thread (mt2);
t2.start();
}
}
```

# LIFE CYCLE OF A THREAD

Q] What is thread? Draw thread life cycle diagram in Java.

[S 17]

Q] Explain thread life cycle diagram in Java.

[W 14, 15] [S 15,16]

During the life time of a thread, there are many states it can enter. They include:

1. Newborn state
2. Runnable state
3. Running state
4. Blocked state
5. Dead state

A thread is always in one of these five states. It can move from one state to another via a variety of ways as shown in Fig.

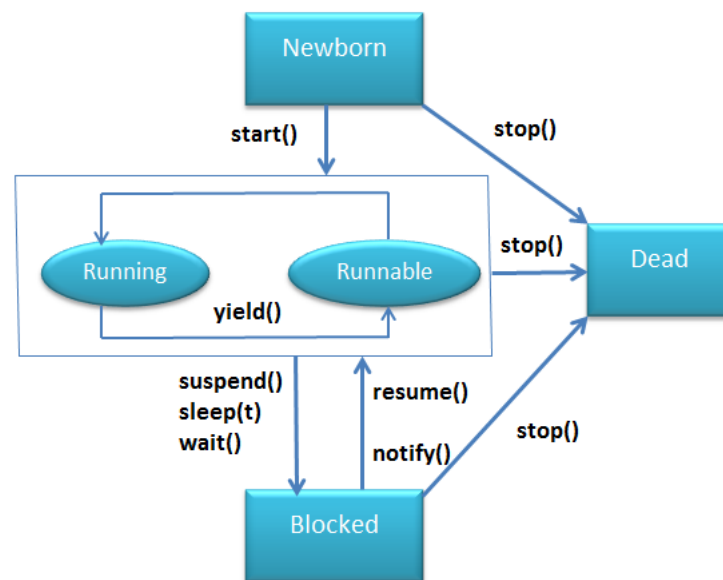


Fig: Life Cycle of Thread

## 1. Newborn state:

When a thread object is created it is said to be in a new born state. When the thread is in a new born state it is not scheduled for running. From this state it can be scheduled for running by `start()` or killed by `stop()`. If it is put in a queue, it moves to runnable state.

## 2. Runnable State:

It means that thread is **ready for execution** and is **waiting for the availability of the processor** i.e. **the thread has joined the queue** and is **waiting for execution**. If all threads have **equal priority** then they are **given time slots** for execution in **round robin** fashion. The thread that **relinquishes control** joins the **queue** at the end and again waits for its turn. A thread can relinquish the control to another by `yield()`.



### 3. Running State:

It means that the **processor** has **given its time** to the **thread for execution**.

The thread runs until it relinquishes control on its own or it is pre-empted by a higher priority thread.

### 4. Blocked state:

A thread can be **temporarily suspended** or **blocked** from entering into the **runnable and running** state by using either of the following thread method.

**suspend()** : Thread can be **suspended** by this method. It can be rescheduled by **resume()**.

**wait()**: If a thread requires to wait until some event occurs, it can be done using **wait** method and can be scheduled to run again by **notify()**.

**sleep(t)**: We can put a thread to sleep for a specified time period using **sleep(time)** where time is in ms. It re-enters the runnable state as soon as **period has elapsed /over** .

### 5. Dead State:

Whenever **we want to stop a thread form running further** we can call its **stop()**.

The statement causes the **thread to move to a dead state**.

A thread will also move to dead state **automatically** when **it reaches to end of the method**.

The stop method may be used when the **premature death** is required.

## STOPPING AND BLOCKING A THREAD

### Stopping a Thread

Whenever we want to stop a thread from running further, we may do so by calling its **stop()** method, like:

```
aThread.stop( );
```

This statement causes the thread to move to the **dead** state.

A thread will also move to the dead state automatically when it reaches the end of its method.

The **stop()** method may be used when the **premature death** of a thread is desired.

### Blocking a Thread

A thread can also be **temporarily suspended or blocked** from entering into the runnable and subsequently running state by using either of the following thread methods:

|                         |                 |                                        |
|-------------------------|-----------------|----------------------------------------|
| <code>sleep( )</code>   | <code>//</code> | blocked for a specified time           |
| <code>suspend( )</code> | <code>//</code> | blocked until further orders           |
| <code>wait ( )</code>   | <code>//</code> | blocked until certain condition occurs |

These methods cause the thread to go into the *blocked (or not-runnable)* state.

The thread will return to the runnable state when the **specified time** is **completed** in the case of **sleep( )**, the **resume( )** method is invoked in the case of **suspend()**, and the **notify()** method is called in the case of **wait()**.

## Thread Control Methods

Q] Explain following methods related to threads.

[4 M] [W 14]

- 1) suspend()
- 2) resume()
- 3) yield()
- 4) wait()

(1 M – each method's syntax and use)

There are various static methods which you can use on thread objects to control their behavior. Following table lists down those methods –

| Sr. | Method & Description                                                                                                                                   |
|-----|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1   | <b>public void suspend()</b><br>This method puts a thread in the suspended state and can be resumed using resume() method.                             |
| 2   | <b>public void stop()</b><br>This method stops a thread completely and moves the thread to Dead State.                                                 |
| 3   | <b>public void resume()</b><br>This method resumes a thread, which was suspended using suspend() method.                                               |
| 4   | <b>public void wait()</b><br>Causes the current thread to wait until another thread invokes the notify().                                              |
| 5   | <b>public void notify()</b><br>To signal a waiting thread that a condition it was waiting for has been met.                                            |
| 6   | <b>public void yield()</b><br>The yield() method causes the currently executing thread object to temporarily pause and allow other threads to execute. |

**Program to show use of sleep method:**

```
class mythread1 extends Thread
{
    public void run()
    {
        try
        {
            int i;
            for(i=1;i<=5;i++)
            {
                Thread.sleep(5000);
                System.out.println("From Thread 1 : "+i);
            }
        }
    }
}
```

```

catch(Exception e)
{
    // catching the exception
    System.out.println(e);
}
}
}
class Sample
{
    public static void main(String args[])
    {
        mythread1 mt1=new mythread1();
        mt1.start();
    }
}

```

Every sleep() method in java throws an exception hence whenever we are using the sleep method compulsory we should handle it either by try-catch or by throws keyword otherwise we will get compile-time error.

### getName() method

The **getName()** method of thread class is used to return the name of thread.

```

class mythread extends Thread
{
    public void run()
    {
        System.out.println("Thread is running...");
    }
    public static void main(String args[])
    {
        // creating two threads
        mythread t1=new mythread();
        mythread t2=new mythread();
        // return the name of threads
        System.out.println("Name of t1: "+ t1.getName()); // Thread - 0
        System.out.println("Name of t2: "+t2.getName()); // Thread - 1
        // start t1 and t2 threads
        t1.start();
        t2.start();
    }
}

```

### Output:-

```

Name of t1: Thread-0
Name of t2: Thread-1
Thread is running...
Thread is running...

```

## currentThread()

The **currentThread()** method of Thread class is used to **return a reference** to the **currently executing thread object**.

### Syntax

```
public static Thread currentThread()
```

```
class mythread extends Thread
```

```
{
    public void run()
    {
        // print currently executing thread
        System.out.println("Current Thread Name is "+Thread.currentThread().getName());
    }
    public static void main(String args[])
    {
        mythread t1=new mythread(); // t1 => Thread 0
        mythread t2=new mythread(); // t2 => Thread 1
        t1.start();
        t2.start();
    }
}
```

## THREAD PRIORITY

**Q] What is thread priority? Write default priority values and methods to change them. [S 17]**

**Q] What is thread priority? How thread priority are set and changed? Explain with example.**

(Thread Priority - 2 marks; each method - 2 marks; Example - 2 marks) [8 M][S 16]

**Thread Priority:** In java each **thread is assigned a priority** which **affects the order in which it is scheduled for running**.

Threads of same priority are given equal treatment by the java scheduler.

**Default Priority Values:** Thread priorities can take value from 1 to 10.

Default Priority Values are :-

**MIN\_PRIORITY = 1**

**NORM\_PRIORITY = 5 (Default Priority)**

**MAX\_PRIORITY = 10**

We can get and set the priority values for threads in Java using the **getPriority()** and **setPriority()** methods. Based on these priorities, the thread scheduler decides the order of thread execution.

We should also remember that there is no guarantee that the scheduler always executes the high priority threads first since it also depends on the JVM implementation.



### 1. setPriority:

**Syntax:** `public void setPriority ( int newPriority);`

`java.lang.Thread.setPriority()` method is used to assign new priority to the thread.

This method throws `IllegalArgumentException` if the value of `newPriority` goes out of Range, which is 1 to 10.

The thread priority can be assigned to thread using `setPriority` method as

**`ThreadName.setPriority (int newPriority);`**

### 2. getPriority:

**Syntax:** `public int getPriority ();`

`java.lang.Thread.getPriority()` method obtain the priority of the thread and returns integer value.

The thread priority can be obtained as :-

**`ThreadName.getPriority();`**

### Example: Program to set and get Priority

class `mythread` extends `Thread`

```
{
    public void run()
    {
        System.out.println(Thread.currentThread().getName()+" is running with Priority "+Thread.currentThread().getPriority());
    }
    public static void main(String args[])
    {
        // creating four threads
        mythread t1 = new mythread();
        mythread t2 = new mythread();
        mythread t3 = new mythread();
        ///mythread t4 = new mythread();

        // print the default priority value of thread
        System.out.println("Default Priority of Threads are :-");
        System.out.println("t1 thread priority : " + t1.getPriority());
        System.out.println("t2 thread priority : " + t2.getPriority());
        System.out.println("t3 thread priority : " + t3.getPriority());
        //System.out.println("t4 thread priority : " + t4.getPriority());

        // Setting thread priority

        t1.setPriority(2);
        t2.setPriority(5);
        t3.setPriority(8);
        //t4.setPriority(MAX_PRIORITY);
    }
}
```



```
System.out.println("Priority of Threads after setPriority are :-");
System.out.println("t1 thread priority : " + t1.getPriority());
System.out.println("t2 thread priority : " + t2.getPriority());
System.out.println("t3 thread priority : " + t3.getPriority());
//System.out.println("t4 thread priority : " + t4.getPriority());

// this will call the run() method
t1.start();
t2.start();
t3.start();
// t4.start();

}
}
```

## ADVANTAGES OF MULTITHREADING

- 1] It enables programmers to do **multiple things at one time**.
- 2] They can **divide a long program** (containing operations that are conceptually concurrent) into **threads** and **execute** them in **parallel**. For example, we can send tasks such as printing into the background and continue to perform some other task in the foreground.
- 3] This approach would considerably **improve** the **speed of our programs**.

# MANAGING ERRORS AND EXCEPTIONS

**Q] What are different types of error? What is use of throw, throws and finally Statement?**

**[Type of Errors & explanation 2-marks, throw, throws & finally 2-mark each] [W 14]**

**Q] What is exception? How it is handled? Explain with suitable example. [W 15]**

**(Definition – 1 Mark, Listing of keywords – 1 Mark, Example – 2 Marks)**

The **problems** in a Java program that **prevent** it from **running normally** are called Java errors.

**Errors** are the **mistakes** that can make a **program go wrong**.

An error may produce an **incorrect output** or **may terminate the execution of the program abruptly** or **even may cause the system to crash**.

It is therefore important to **detect** and **manage** properly all the **possible error conditions** in the program so that the **program will not terminate or crash** during execution.

## TYPES OF ERRORS

1. Compile time errors
2. Runtime errors
3. Logical Error

### 1. Compile time errors:

Compile Time Errors are those errors which prevent the code from running because of an incorrect syntax such as a missing semicolon at the end of a statement or a missing bracket etc. These errors are **detected** by the **java compiler** at **compile time** and an **error message** is displayed on the screen while compiling.

Compile Time Errors are sometimes also referred to as **Syntax errors**.

Java compiler **does not produce .class file** until these errors are resolved.

So it is necessary to fix all the errors before the successful compilation of program.

**Example:**

```
class Error1
{
    public static void main (String args[])
    {
        System.out.println("Hello Java")
    }
}
```

After compiling above code JAVA compiler displays following error:

```
Error1.java : ';' Expected
System.out.println("Hello Java")
                        ^
1 error
```

### The most of common compile errors are:

- Missing semicolon
- Missing (or mismatch of) bracket in classes & methods
- Misspelling of identifiers & keywords
- Missing double quotes in string
- Use of undeclared variables.

### 2. Runtime errors:

Runtime errors occur when the program has successfully compiled without giving any errors and creating a ".class" file. But, the program does not execute properly.

These errors are detected by **Java Virtual Machine(JVM)** at **runtime** or at the time of execution of the program.

The **Java compiler** does **not detect** runtime errors because the Java compiler does not have any technique to catch these errors.

These runtime errors are called **exceptions** and they terminate the program abnormally, giving an error statement.

#### For example:

1. Dividing an integer by zero.
2. Trying to access an element that is out of range of the array.
3. If the user inputs a data of string format when the computer is expecting an integer, there will be a runtime error.

#### Example : Runtime Error caused by dividing by zero

```
class Main {  
    public static void main(String args[])  
    {  
        int var1 = 15;  
        int var2 = 0;  
        int ans1 = var1 / var2;  
        // This statement causes a runtime error,  
        // as 15 is getting divided by 0 here  
        System.out.println("Division is: "+ ans1);  
    }  
}
```

#### Output:-

```
Exception in thread "main" java.lang.ArithmeticException: / by zero  
    at Main.main(Main.java:6)
```

### 3. Logical Error

A logic error is when your program compiles and executes, but returns an incorrect result.

**Logical Error** are neither detected by the Java compiler nor by the JVM.

The programmer is entirely responsible for them.

Logical errors are also called **Semantic Errors**.

These are the **hardest** to find because the code runs without crashing, but the output is wrong.

Example: The programmer wants to print even numbers but, instead of using a modulus operator(%) but he/she uses a divide operator (/).

```
class JavaError
{
    public static void main(String[] args)
    {
        for (int i = 0; i <= 10; i++)
        {
            if (i / 2 == 0)           // it should be % in place of /
            {
                System.out.println(i);
            }
        }
    }
}
```

**Actual Output :**

0

1

**Expected Output :**

0

2

4

6

8

10





## ➤ EXCEPTIONS

### Q] Define exception. State built-in exceptions.

An exception is an event, which occurs during the execution of a program, that disrupts the normal flow of the program execution.

An exception is a condition that is caused by the runtime errors in the program.

#### Example:

```
class Main {  
    public static void main(String args[])  
    {  
        int var1 = 15;  
        int var2 = 0;  
        int ans1 = var1 / var2;  
        // This statement causes a runtime error as 15 is getting divided by 0  
        System.out.println("Division is: " + ans1);  
    }  
}
```

Above program is syntactically correct and does not produce any compile time error.

But division by zero is runtime error.

So while executing, it displays the following message and stops without executing further statements.

#### Output:-

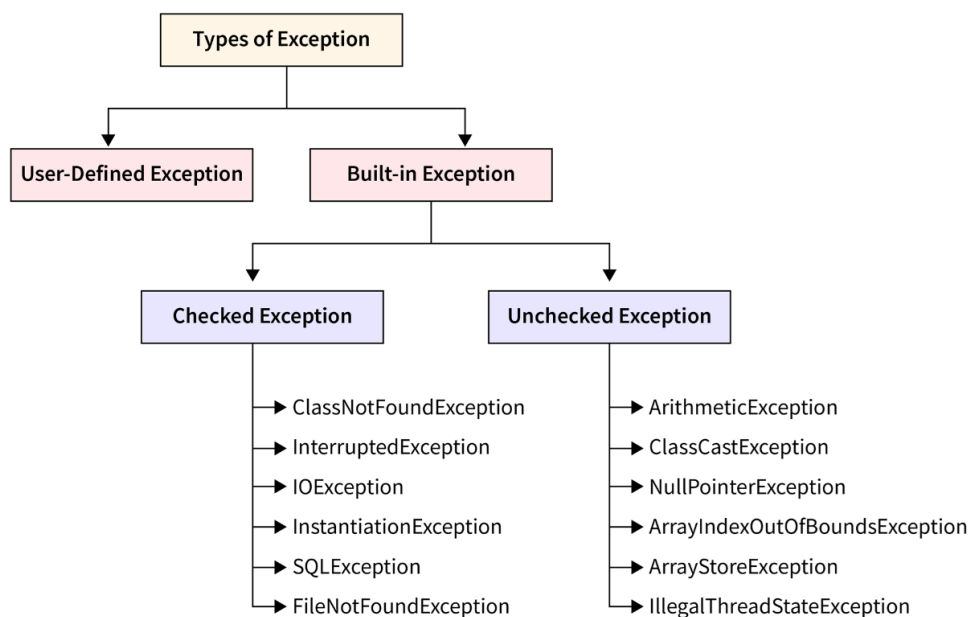
Exception in thread "main" java.lang.ArithmeticException: / by zero  
at Main.main(Main.java:6)

Exceptions can be categorized into two ways:

#### 1. Built-in Exceptions

- Checked Exception
- Unchecked Exception

#### 2. User-Defined Exceptions



## ➤ Built-in exceptions:

Exceptions that are already available in **Java libraries** are referred to as **built-in exception**.

- **ArithmeticException:** It is thrown when an **exceptional condition** has occurred in an **arithmetic operation**.
- **ArrayIndexOutOfBoundsException:** It is thrown to indicate that an array has been accessed with an **illegal index**. The index is either **negative** or **greater than or equal to the size of the array**.
- **ClassNotFoundException:** This Exception is raised **when we try to access a class** whose **definition is not found**.
- **FileNotFoundException:** This Exception is raised when we try to access a **nonexistent file**.
- **IOException:** It is thrown when an **input-output operation failed or interrupted**.
- **NullPointerException:** This exception is raised when **referring to the members of a null object**. Null represents nothing.
- **NumberFormatException:** Caused when a **conversion** between **strings** and **number** fails.
- **StringIndexOutOfBoundsException:** It is thrown by String class methods to indicate that an index is either **negative or greater than the size of the string**.
- **OutOfMemoryException:** Caused when **there's not enough memory to allocate a new object**.
- **StackOverflowException:** Caused when the system **runs out of stack space**.
- **IllegalArgumentException :** Passing parameter that is **not in a valid range** or value for method.

## ➤ Checked Exceptions

Exceptions that occur and can be **detected at compile time** are known as **Checked Exceptions**.

These exceptions **prevent the program from running**, and the **compiler forces you to handle these exceptions** explicitly.

Otherwise, the program will not get compiled.

eg :- sleep() method in Thread class must be enclosed in try catch block otherwise it throws InterruptedException.

**Examples of Checked Exception:-**

ClassNotFoundException, FileNotFoundException, IOException, SQLException, etc.

## ➤ Unchecked Exceptions

Exceptions that occur at the **time of the program execution, i.e., at runtime**, are known as **Unchecked Exceptions**. These exceptions are ignored at the time of compilation.

**Examples:-**

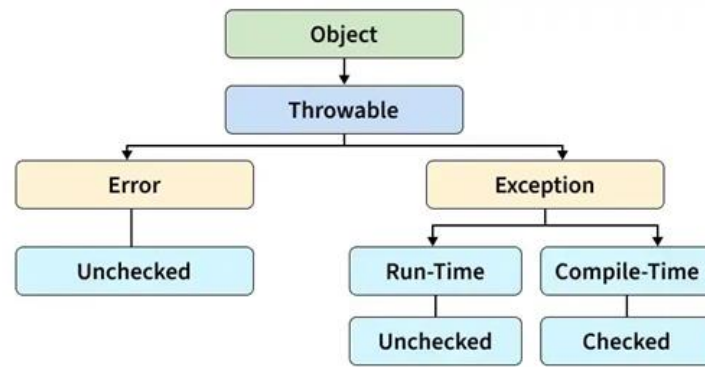
ArrayIndexOutOfBoundsException, NullPointerException, IllegalArgumentException, ArithmeticException, NumberFormatException, etc.

All **exception types** are **subclasses** of the built-in class **Throwable**.

Both **java.lang.Error** and **java.lang.Exception** classes are sub classes of **java.lang.Throwable** class.

**java.lang.Error** class represents the errors which are mainly caused by the **environment** in which **application** is running.

Where as **java.lang.Exception** class represents the **exceptions** which are **mainly caused by the application itself**.



| Exception                                                                                                   | Error                                                                             |
|-------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| You can <b>recover from</b> exception by handling them through <b>try catch block</b> .                     | It is <b>impossible to recover</b> from error.                                    |
| Exception can be classified into two types:<br><b>a. Checked Exception</b><br><b>b. Unchecked Exception</b> | There is no such classification for errors.<br>Errors are always <b>unchecked</b> |
| Can occur at <b>runtime</b> or <b>compile time</b> due to an <b>issue in the program</b>                    | Occur at <b>runtime</b> due to <b>lack of system recourses</b> .                  |
| Exceptions in java are of type <code>java.lang.Exception</code> .                                           | Errors in java are of type <code>java.lang.Error</code>                           |
| e.g. <code>ArithmeticException</code><br><code>SQLException</code>                                          | e.g. <code>OutOfMemoryError</code><br><code>StackOverflowError</code>             |

ARROW COMPUTER

# Exception Handling

An exception is an event, which occurs during the execution of a program, that disrupts the normal flow of the program execution.

Exception handling is a mechanism to handle runtime errors, allowing the normal flow of a program to continue.

When the java interpreter encounters errors such as dividing an integer by zero, it creates an **exception object** and **throws it** (i.e informs us that error has occurred.)

If the exception object is not caught and handled properly, the interpreter will display an error message and will terminate the program.

If we want the program to continue with the execution of the remaining code, then we should attempt to catch the exception object thrown by the error condition and then display an appropriate message for taking corrective actions.

This task is known as **exception handling**.

The purpose of exception handling mechanism is to detect and report an “exceptional circumstance” so that appropriate action can be taken.

The mechanism suggests following tasks:

1. Find the problem (**Hit** the exception).
2. Inform that an error has occurred (**Throw** the exception)
3. Receive the error information (**Catch** the exception)
4. Take corrective actions (**Handle** the exception)

The error handling code basically consists of two segments,

1. To detect errors and to throw exceptions
2. To catch exceptions and to take appropriate actions.

**Q] Define an exception. How it is handled?**

**(Definition – 1 M, explanation with all keywords – 3 M)**

**Q] Explain the following clause w.r.t. exception handling: [W 15]**

- (i) try
- (ii) catch
- (iii) throw
- (iv) finally

Java handles exceptions with 5 keywords:

1) try 2) catch 3) finally 4) Throw 5) throws

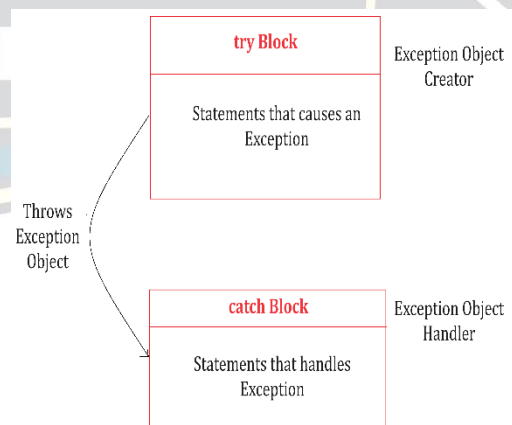
## 1. try-

Program statements that **you want to monitor for exceptions** are contained within a **try block**. If an exception occurs within the try block, it is thrown.

**Syntax:**

```
try
{
// statement(s) that might cause exception
}
```

Note: While writing a program, if you think that certain statements in a program can throw an exception, enclose them in try block and handle that exception.



Mechanism of exception handling



## 2. Catch block

A catch block is used to **handle the exceptions**. This block **must follow the try block**.

A **single try block** can have **several catch blocks associated** with it.

You can catch different exceptions in different catch blocks.

When an exception occurs in try block, the corresponding catch block that handles that particular exception executes.

For example if an arithmetic exception occurs in try block then the statements enclosed in catch block for arithmetic exception executes.

### Syntax:

```
catch (ExceptionType1 exOb)
```

```
{
```

```
    // exception handler for ExceptionType1
```

```
}
```

**The general form of an exception handling block is**

```
try
```

```
{
```

```
    // block of code to monitor for errors
```

```
}
```

```
catch (ExceptionType1 e)
```

```
{
```

```
    // exception handler for ExceptionType1
```

```
}
```

```
catch (ExceptionType2 e)
```

```
{
```

```
    // exception handler for ExceptionType2
```

```
}
```

```
finally
```

```
{
```

```
    // block of code to be executed after try block ends
```

```
}
```

**Example to show use of try and catch blocks to handle arithmetic exception:**

```
class Error3
```

```
{
```

```
public static void main(String args[])
```

```
{
```

```
int a = 11, b = 5, c = 5;
```

```
try
```

```
{
```

```
int x = a / (b - c); // Division by zero
```

```
System.out.println("x=" + x);
```

```
}
```

```
catch (Exception e)
```

```
{
```

```
System.out.println("Division by zero");
```

```
}
```

```
int y=a/(b+c);
System.out.println("y="+y);
}
}
```

### Output

Division by zero

y=1

### Internal working of try-catch Block

If an exception occurs in try block then the control of execution is passed to the corresponding catch block.

A single try block can have multiple catch blocks associated with it.

You should place **generic catch block at the end**.

A generic catch block can handle all the exceptions, whether it is

**ArrayIndexOutOfBoundsException** or **ArithmeticException** or **NullPointerException** or any other type of exception.

If you place it before all the catch blocks then it will display the generic message.

You always want to give the user a meaningful message for each type of exception rather than a generic message.

```
class Error3
{
    public static void main(String args[])
    {
        int a =11,b=5,c = 5;
        try
        {
            int x = a/(b-c); //Division by zero
            System.out.println("x="+x);
        }
        catch(ArithmeticException e)
        {
            System.out.println("Division by zero");
        }
        catch(Exception e)
        {
            System.out.println("Exception Occured");
        }
        int y=a/(b+c);
        System.out.println("y="+y);
    }
}
```

### Output:

Division by zero

y= 1

### Why we need other catch handlers when we have a generic that can handle all?

This is because in generic exception handler you can display a message but you are not sure for which type of exception it may trigger so it will display the same message for all the exceptions and user may not be able to understand which exception occurred.

Thats the reason you should place it at the end of all the specific exception catch blocks

catch(ArithmeticException e) is a catch block that can handle ArithmeticException

catch(NullPointerException e) is a catch block that can handle NullPointerException

### Example of Multiple catch blocks

```
class Sample
```

```
{
    public static void main(String[] args)
    {
        int[] arr = new int[10];
        try
        {
            arr[10] = 11;
        }

        catch (NumberFormatException e1)
        {
            System.out.println("NumberFormatException");
        }
        catch(ArithmeticException e)
        {
            System.out.println("Arithmetic Exception");
        }
        catch (ArrayIndexOutOfBoundsException e2)
        {
            System.out.println("You are trying to access array Element out of range");
        }
        catch(Exception e)
        {
            System.out.println("Error Occured");
        }

        System.out.println("Final Statement after Try and Catch Block");
    }
}
```

### Output:

**You are trying to access array Element out of range**

**Final Statement after Try and Catch Block**

### 3. finally

finally block is used to execute important code such as closing connection, stream etc.

Java finally block is always executed whether exception is handled or not.

Java finally block follows try or catch block.

#### Syntax of Finally block

```
try
{
    //Statements that may cause an exception
}
catch
{
    //Handling exception
}
finally
{
    // block of code to be executed before try block ends
}
```

#### A Simple Example of finally block

Here you can see that the exception occurred in try block which has been handled in catch block, after that finally block got executed.

class Example

```
{
    public static void main(String args[]) {
        try
        {
            int num=121/0;
            System.out.println(num);
        }
        catch(ArithmeticException e)
        {
            System.out.println("Number should not be divided by zero");
        }
        finally
        {
            System.out.println("This is finally block");
        }
        System.out.println("Out of try-catch-finally");
    }
}
```

#### Output:

Number should not be divided by zero

This is finally block

Out of try-catch-finally



## Example 2:-

In this case, the program throws an exception but not handled by catch so finally block execute after the try block and after the execution of finally block program terminate abnormally, But finally block execute fine.

```
class Test
{
    public static void main(String args[]) {
        try
        {
            int num=121/0;
            System.out.println(num);
        }
        catch(NullPointerException e)
        {
            System.out.println("Number should not be divided by zero");
        }
        /* Finally block will always execute even if there is no exception in try block */
        finally
        {
            System.out.println("This is finally block");
        }
        System.out.println("Out of try-catch-finally");
    }
}
```

### Output:-

```
This is finally block
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at Test.main(Test.java:5)
```

### Few Important points regarding finally block

1. A finally block must be associated with a try block, you cannot use finally without a try block. You should place those statements in this block that must be executed always.
2. Finally block is **optional**, a try-catch block is sufficient for exception handling, however if you place a finally block then it will always run after the execution of try block.
3. In normal case when there is no exception in try block then the finally block is executed after try block.  
However if an exception occurs then the catch block is executed before finally block.
4. An exception in the finally block, behaves exactly like any other exception.
5. The statements present in the **finally block** execute even if the try block contains control transfer statements like return, break or continue.

## 4. throw

The "throw" keyword is used to **explicitly** throw an exception from a **method** or **any block of code**.

We can throw either **checked** or **unchecked exceptions** in java by throw keyword.

The "throw" key-word is mainly used to throw a custom (**user defined**) exception.

The general form of throw is:

**throw new ThrowableInstance;**

**ThrowableInstance** must be an object of the type Throwable or a subclass of Throwable.

When an exception is thrown, the flow of program execution transfers from the try block to the catch block.

Any subsequent statements are not executed.

We use the throw keyword within a method.

### Example : Java throw keyword

Throw an exception if **age** is below 18 (print "Access denied"). If age is 18 or older, print "Access granted":

```
class Example1
{
    void checkAge(int age)
    {
        if (age < 18)
        {
            throw new ArithmeticException("Access denied - You must be at least 18 years old.");
        }
    }
    else
    {
        System.out.println("Access granted - You are old enough!");
    }
}

public static void main(String[] args)
{
    Example1 e = new Example1();
    e.checkAge(10);
}
```

## 5. throws

The throws keyword is used to **declare the list of exception** that a **method may throw** during execution of program.

Any method that is capable of causing exceptions must **list** all the exceptions possible during its execution, so that caller of method will know how to handle exceptions.

A method can do so by using the **throws** keyword.

### Syntax:

```
type method_name(parameter_list) throws exception_list
{
    // definition of method
}
```

### Example throws Keyword

Here, we have a method that can throw Arithmetic exception so we mentioned that with the method declaration and catch that using the catch handler in the main method.

```
class Test
{
    static void check() throws ArithmeticException
    {
        System.out.println("Inside check function");
        throw new ArithmeticException("demo");
    }

    public static void main(String args[])
    {
        try
        {
            check();
        }
        catch(ArithmeticException e)
        {
            System.out.println("caught" + e);
        }
    }
}
```

Inside check function

caught java.lang.ArithmeticException: demo

### Difference between throw and throws

| throw                                                                                                 | throws                                                                                              |
|-------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| throw keyword is used to <b>throw an exception</b> explicitly.                                        | throws keyword is used to <b>declare an exception</b> possible during its execution.                |
| throw keyword is followed by an <b>instance of Throwable class</b> or one of its <b>sub-classes</b> . | throws keyword is followed by one or more <b>Exception class names</b> separated by <b>commas</b> . |
| throw keyword is declared inside a method body.                                                       | throws keyword is used with <b>method signature</b> (method <b>declaration</b> ).                   |
| We cannot throw multiple exceptions using throw keyword.                                              | We can <b>declare multiple exceptions</b> (separated by commas) using throws keyword.               |

## Drawback of Scanner:-

In Scanner class if we call **nextLine()** method after any one of the seven **nextXXX()** method then the nextLine() does not read values from console and will skip that step.

nextXXX() methods ignore newline character and nextLine() only reads till the first newline character.

The nextXXX() methods are nextInt(), nextFloat(), nextByte(), nextShort(), nextDouble(), nextLong(), next().

```
import java.util.*;
class Sample
{
public static void main(String args[])
{
    Scanner sc = new Scanner(System.in);
    System.out.println("Enter roll and name");
    int roll=sc.nextInt();
    String name=sc.nextLine();
    System.out.println("Roll is "+roll);
    System.out.println("Name is "+name);
}
}
```

### Output:-

Enter roll and name

4

Roll is 4

Name is



# BufferedReader

Java `BufferedReader` class of the `java.io` package is used to **read the text** from a **character-based input stream**.

It can be used to read **data line by line** by `readLine()` method.

It makes the **performance fast**.

It inherits **Reader** class.

```
BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
```

- **System.in** is an **InputStream** that **connects directly to the standard input** (the **keyboard**).
- It reads raw data as **bytes**, not characters.
- The **InputStreamReader** class of the **java.io** package convert **bytes** into **characters**.
- The `BufferedReader` wraps the `InputStreamReader` and stores characters in an internal memory buffer.
- `readLine()` scans characters in the internal buffer line by line.

## Note:-

The `readLine()` method of `BufferedReader` throws an `IOException` when there is a problem reading. Programs that use `BufferedReader` for input need to do something about possible exceptions. So we need to declare `IOException` after method declaration in which `readLine()` method is used.

## Reading data from console by `InputStreamReader` and `BufferedReader`

### Example 1:

```
import java.io.*;
class Sample
{
    public static void main(String args[]) throws IOException
    {
        InputStreamReader r=new InputStreamReader(System.in);
        BufferedReader br=new BufferedReader(r);
        System.out.println("Enter your name");
        String name=br.readLine();
        System.out.println("Welcome "+name);
    }
}
```

**Exapmle 2:**

```
import java.io.*;
class Sample
{
    public static void main(String args[])throws IOException
    {
        BufferedReader br=new BufferedReader(new InputStreamReader(System.in));
        System.out.println("Enter your name");
        String name=br.readLine();
        System.out.println("Welcome "+name);
    }
}
```

**Example 3:**

```
import java.io.*;
class Employee
{
    public static void main(String args[]) throws IOException
    {
        BufferedReader br =new BufferedReader(new InputStreamReader(System.in));
        System.out.println("Enter your name: ");
        String name = br.readLine();
        System.out.println("Enter your age: ");
        int age = Integer.parseInt(br.readLine());
        System.out.println("Enter your Id: ");
        int id = Integer.parseInt(br.readLine());
        System.out.println("Name: "+name);
        System.out.println("Age: "+age);
        System.out.println("Id: "+id);
    }
}
```

## User-defined Exception:

Sometimes the programmer wants to create his **own customized exception** as per the **requirements of the application** which is called **user-defined exception** or **custom exception**.

Custom exceptions in Java are those exceptions that are created by a programmer to meet the specific requirements of the application. That's why it is also known as user-defined exception in Java.

### For example:

1. A banking application, a customer whose age is less than 18 years, the program throws a custom exception indicating "needs to open joint account".
2. Voting age in India: If a person's age entered is less than 18 years, the program throws "invalid age" as a custom exception.

### How to create your own User-defined Exception in Java?

**Step 1:** User-defined exceptions can be created simply by extending Exception class. This is done as:

```
class OwnException extends Exception
```

**Step 2:** Provide **constructors** to initialize the exception with custom messages.

This can be done as follows:

```
OwnException (String m)  
{  
    super(m);  
}
```

**Step 3:** Define try block and then give call to user-defined exception (constructor) using throw clause.

```
throw new OwnException();
```

**Step4:** Handle the exception in catch block.

**Q) Write a program to input name and age of a person and throws an user define exception if entered age is negative.** **[S 15, S 16]**

```
import java.io.*;
class NegativeAgeException extends Exception
{
    NegativeAgeException(String str)
    {
        super(str);
    }
}

public class Main
{
    public static void main(String[] args) throws IOException
    {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        System.out.println("Enter Name and Age");
        String name = br.readLine();
        int age = Integer.parseInt(br.readLine());

        try
        {
            if(age<0)
            {
                throw new NegativeAgeException("Age can not be Negative");
            }
            else
            {
                System.out.println("You have entered Valid Details");
            }
        }
        catch(NegativeAgeException e)
        {
            System.out.println(e);
        }
    }
}
```

**Q] Write a program to accept password from user and throw 'Authentication failure' exception if password is incorrect. [W 15] (Correct logic - 5 Marks, for syntax - 3 Marks)**  
**Using BufferedReader**

```
import java.io.*;
class PasswordException extends Exception
{
    PasswordException(String str)
    {
        super(str);
    }
}
class Sample
{
    public static void main(String args[]) throws IOException
    {
        BufferedReader br=new BufferedReader(new InputStreamReader(System.in));
        System.out.println("Enter Password : ");
        String S1=br.readLine();
        try
        {
            if(S1.equals("EXAMW15"))
            {
                System.out.println("Authenticated ");
            }
            else
            {
                throw new PasswordException("Authentication Failure");
            }
        }
        catch(PasswordException e)
        {
            System.out.println(e);
        }
    }
}
```



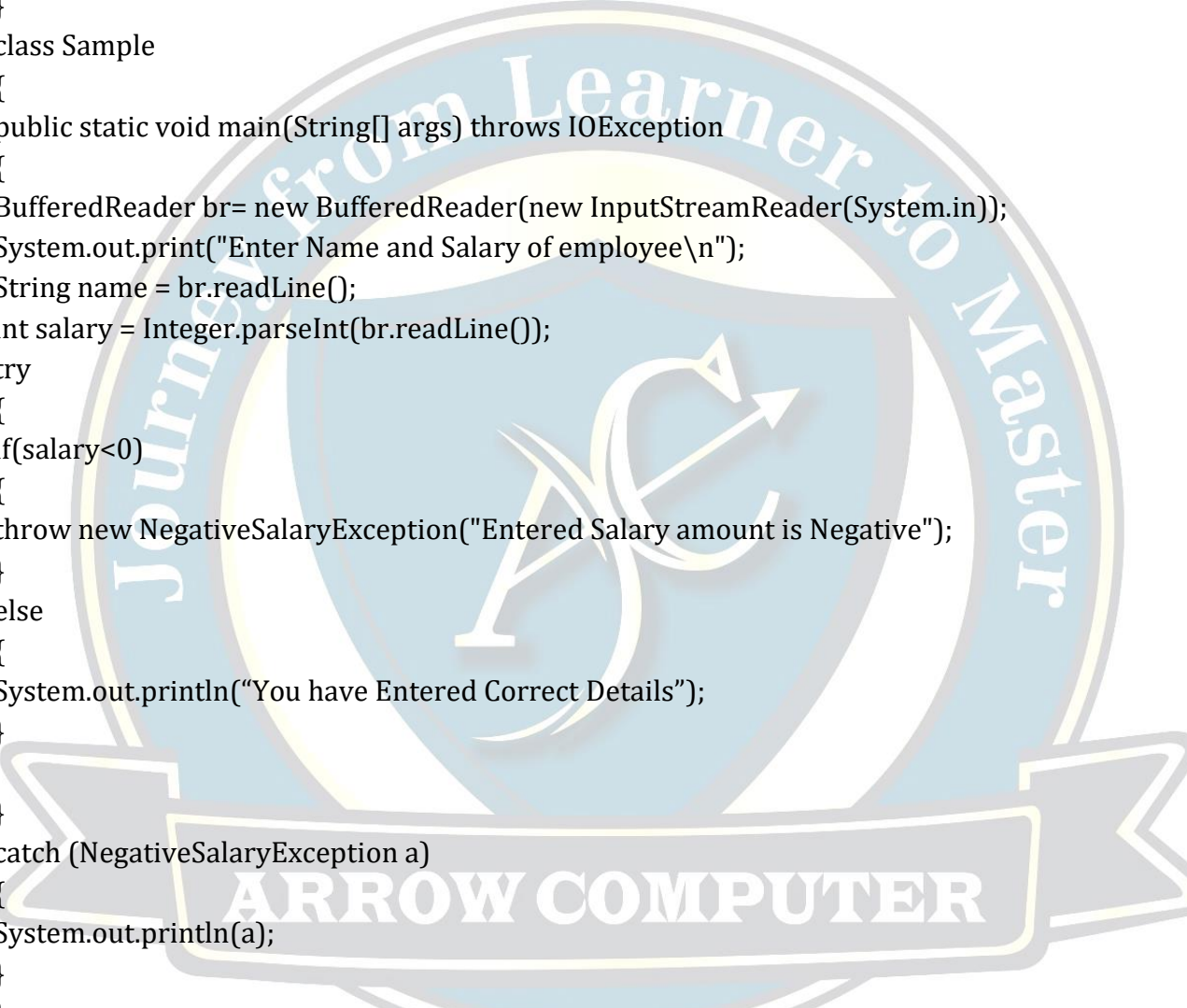
**Q] Define an exception called “No match Exception” that is thrown when a string is not equal to “MSBTE”. Write program.** **[W 16]**

```
import java.io.*;
class NoMatchException extends Exception
{
    NoMatchException(String str)
    {
        super(str);
    }
}
class Sample
{
    public static void main(String args[]) throws IOException
    {
        BufferedReader br= new BufferedReader(new InputStreamReader(System.in) );
        System.out.println("Enter a String:");
        String S1= br.readLine();
        try
        {
            if(S1.equals("MSBTE"))
            {
                System.out.println("Strings are equal");
            }
            else
            {
                throw new NoMatchException("Strings are not equal");
            }
        }
        catch(NoMatchException e)
        {
            System.out.println(e);
        }
    }
}
```



**Write a program to input name and salary of employee and throw user defined exception if entered salary is negative.**

```
import java.io.*;
class NegativeSalaryException extends Exception
{
    NegativeSalaryException(String str)
    {
        super(str);
    }
}
class Sample
{
    public static void main(String[] args) throws IOException
    {
        BufferedReader br= new BufferedReader(new InputStreamReader(System.in));
        System.out.print("Enter Name and Salary of employee\n");
        String name = br.readLine();
        int salary = Integer.parseInt(br.readLine());
        try
        {
            if(salary<0)
            {
                throw new NegativeSalaryException("Entered Salary amount is Negative");
            }
            else
            {
                System.out.println("You have Entered Correct Details");
            }
        }
        catch (NegativeSalaryException a)
        {
            System.out.println(a);
        }
    }
}
```



## SYNCHRONIZATION

Q] What is synchronization? When do we use it? Explain synchronization of two threads.  
[W 16]

### Synchronization:-

Threads use their **own data** and **methods** provided inside **their run()** methods.

But if we **wish to use data** and **methods outside** the **thread's run()** method, they may **compete** for the **same resources** and **may lead to serious problems**.

When **two or more threads** need **access to a shared resource**, they need some way to **ensure** that the **resource will be used by only one thread at a time**.

The process by which this is achieved is called **synchronization**.

For ex.: One thread may try to read a record from a file while another is still writing to the same file.

When the method is declared as **synchronized**, Java creates a "monitor" and hands it over to the thread that calls the method first time.

### synchronized (lock-object)

```
{  
    ..... // code here is synchronized  
}
```

Synchronization used when we want to -

- 1) prevent data corruption.
- 2) prevent thread interference.
- 3) Maintain consistency

### Program based on synchronization:

```
class Table  
{  
    synchronized void printTable(int n) //synchronized method  
{  
        for(int i=1;i<=10;i++)  
        {  
            System.out.println(n*i);  
            try  
            {
```

```

        Thread.sleep(400);
    }
    catch(Exception e)
    {
        System.out.println(e);}
    }
}

```

```

class MyThread1 extends Thread
{
    Table t;
    MyThread1(Table t)
    {
        this.t=t;
    }
    public void run()
    {
        t.printTable(5);
    }
}

```

```

class MyThread2 extends Thread
{
    Table t;
    MyThread2(Table t)
    {
        this.t=t;
    }
    public void run()
    {
        t.printTable(100);
    }
}

```

```

public class Sample
{
    public static void main(String args[])
    {
        Table obj = new Table();//only one object
        MyThread1 t1=new MyThread1(obj);
        MyThread2 t2=new MyThread2(obj);
        t1.start();
        t2.start();
    }
}

```

